Data Structures and Algorithms -3 (25CS2103E)

Mini Search Engine/ Information-Retrieval System

Team Members

B.Deepak - 2520080035

A.Lakshmi Sai Teja - 2520080005

Under the Esteemed Guidance of

Dr.Anitha Cherkuri
Assistant Professor

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

This project presents the design and implementation of a Mini Search Engine and Information-Retrieval System using Java, developed from first principles without relying on external search libraries. The system efficiently indexes a collection of documents and supports multiple types of searches, including exact keyword matching, phrase and substring searching, fuzzy matching, and document-similarity ranking. An inverted index is used for fast keyword retrieval, while suffix arrays support efficient substring and phrase searches. Levenshtein distance using dynamic programming is implemented to handle fuzzy searches and identify similar words. For relevance-based ranking, the system uses TF-IDF vectorization and cosine similarity to determine the similarity between the query and documents. The project focuses on understanding the core algorithms, data structures, time and space complexities, and performance trade-offs involved in building a search engine.

Introduction

- **Information Retrieval (IR)** is the process of finding relevant information from a collection of documents.
- Search engines use indexing, searching, and ranking techniques to retrieve useful information efficiently.
- This project develops a **custom Mini Search Engine using Java** from first principles.
- The system does not depend on external search libraries for its core search operations.
- It supports four major search types:
  - Exact keyword search
  - Phrase and substring search
  - Fuzzy string search
  - Document similarity ranking
- Different algorithms and data structures are used for different types of queries.

Literature Survey

Existing Systems	Our System
Often depend on powerful external libraries	Algorithms implemented from first principles
Focus mainly on practical search performance	Focuses on algorithmic understanding + retrieval performance
Individual techniques are often studied separately	Integrates multiple advanced algorithms in one search engine
Limited transparency of internal implementation	Complete control over data structures and algorithms
Large-scale systems are difficult to reproduce for students	Small, reproducible and educational system
May require complex NLP/ML infrastructure	Uses classical and advanced algorithms from the syllabus

Problem Statement and Objectives

Problem Statement

Simple keyword searching cannot effectively handle spelling mistakes, phrases, substrings, and document similarity.

Using only one search technique may produce incomplete or less relevant results.

Therefore, a system is required that can support different types of queries using efficient algorithms and data structures.

Objectives

To develop a Mini Search Engine using Java.

To implement an Inverted Index for exact keyword searching.

To implement Suffix Arrays for phrase and substring retrieval.

To implement Levenshtein Distance for fuzzy string matching.

To implement TF-IDF and Cosine Similarity for document ranking.

To analyze the time and space complexity of the implemented algorithms.

Results

The Mini Search Engine successfully supports four major query types:

1. Exact Keyword Search

Uses the Inverted Index.

Quickly retrieves documents containing the searched keyword.

2. Phrase / Substring Search

Uses the Suffix Array.

Finds documents containing specific phrases or substrings.

3. Fuzzy Search

Uses Levenshtein Distance with dynamic programming.

Finds similar words even when the user makes spelling mistakes.

4. Similarity Ranking

Uses TF-IDF and Cosine Similarity.

Ranks documents according to their relevance to the search query.

Hardware and Software requirements

Hardware Requirements:

- Processor: Intel Core i3 or above
- RAM: 4 GB or above
- Storage: 10 GB or above
- Keyboard and Mouse
- Monitor

Software Requirements:

- Operating System: Windows Programming Language: Java
- JDK: Java Development Kit
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Data Structures: HashMap, HashSet, ArrayList, TreeMap
- Concepts: Inverted Index, Searching, Sorting, Ranking
- Java Libraries: java.util, java.io, java.nio

Conclusion

- The project successfully implements a custom Mini Search Engine using Java.
- Four major search capabilities are supported: exact, substring/phrase, fuzzy, and similarity-based search.
- The Inverted Index provides efficient keyword retrieval.
- Suffix Arrays support phrase and substring searching.
- Levenshtein Distance handles fuzzy matching and spelling variations.
- TF-IDF and Cosine Similarity provide relevance-based document ranking.
- The project demonstrates the practical application of Java, Data Structures, and Algorithms in Information Retrieval.
- It also provides an understanding of time complexity, space complexity, and performance trade-offs in search systems.

Future Scope

- Add Natural Language Processing (NLP) for better query understanding.
- Implement advanced spelling correction and autocomplete.
- Improve ranking accuracy using advanced information-retrieval algorithms.
- Support larger document collections and scalable indexing.
- Optimize fuzzy searching for faster retrieval.
- Add support for multiple languages.
- Develop a web-based graphical user interface.
- Introduce machine-learning-based document ranking.
- Improve real-time search performance.

(DEEMED TO BE UNIVERSITY)

Thank you